CS5200 Final Report

Jessica Pham, Shubham Madhusudan Hadawle
Group: PhamJHadawleS

March 12, 2026

1 Project Description

We would like to propose a cocktail database with which a user can look up recipes of a specific name, flavor(s), liquor type, number of ingredients, or difficulty through a website we will call MixMaster. The user should also be able to favorite cocktails and post reviews. Each cocktail entry will display a list of ingredients with quantities, tools needed, preparation instructions, flavor tags, recommended glass type, and a photo. The application distinguishes between "simple" (where ingredients are merely combined) and "complex" cocktails (which require a specific sequence of preparation steps).

We chose this topic because we wanted to have a little fun! The primary goal is to provide an intuitive, searchable interface for beginners looking for easy recipes as well as for bartenders exploring new combinations.

1.1 Tech Stack

Frontend — React 18, Vite, D3.js, Lucide Icons
Backend — FastAPI, SQLAlchemy, PyMySQL
Database — MySQL 8.0
Styling — Inline CSS, Google Fonts (Playfair Display, Outfit)

1.2 Application Functionalities

- Users should be able to see ingredients and flavors for each cocktail.
- Users should be able to login/register and favorite cocktails.
- Users should be able to post reviews for cocktails.
- Users should be able to query cocktails based on flavors and if its difficulty is simple or complex.
  - "simple" cocktails just involve mixing of all ingredients
  - "complex" cocktails require a specific sequence of steps
- Users should be able to query based on fewest/most number of ingredients.
- Users should be able to query based on liquor(s) (ingredient type)
- Each cocktail should have a picture and instructions along with a list of tools that cocktail needs.
- Each cocktail should also show which glass type is usually used for that cocktail if applicable.

2 Database Design

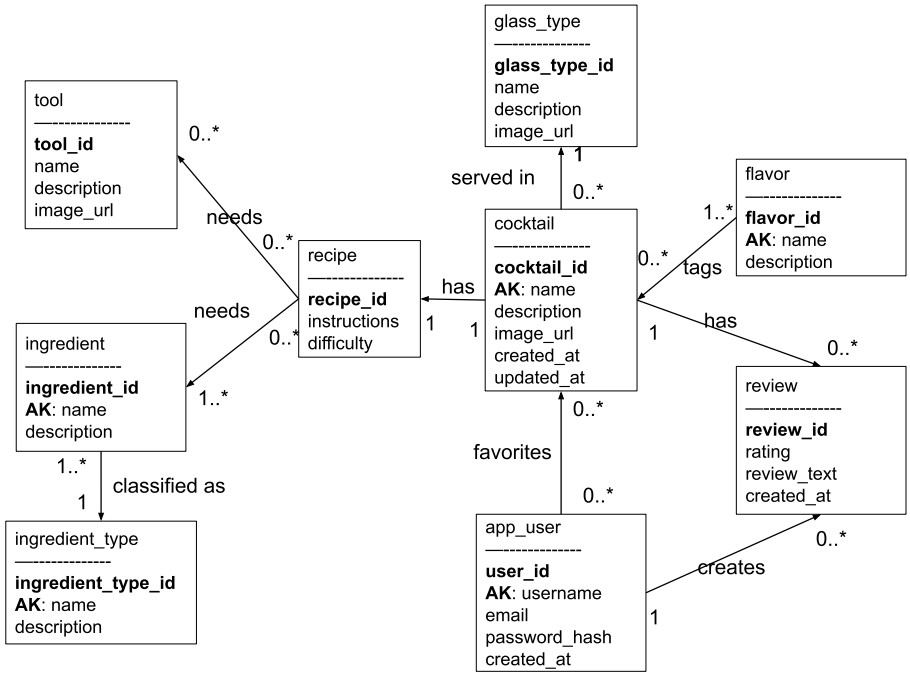

Figure 1: UML Diagram

2.1 Entities and Attributes

The database is organized around the following core entities:

Cocktail: The central entity. Each cocktail has a unique identifier, a name, a text description, a difficulty classification (simple or complex), an image URL, and timestamp fields for record creation and modification. Each cocktail is associated with exactly one glass type.

GlassType: Represents the style of glassware recommended for serving a cocktail (e.g., Coupe, Highball, Rocks). Each glass type has a unique identifier, a name, a description, and an optional image URL.

Recipe: Each Cocktail has exactly one recipe which contains instructions and describes the difficulty of the recipe. The recipe lists the tools and ingredients needed to make the cocktail.

Ingredient: Represents an individual ingredient that may appear in many recipes. Each ingredient has a unique identifier, a name, an optional description, and a foreign key linking it to its ingredient type.

IngredientType: Categorizes ingredients into groups such as Spirit, Mixer, Garnish, Syrup, Juice, or Bitters. This entity enables users to filter cocktails by liquor (i.e., ingredients of type "Spirit"). Each type has a unique identifier, a name, and a description.

Flavor: Represents a flavor profile tag (e.g., Sweet, Sour, Bitter, Smoky, Herbal). Each flavor has a unique identifier, a name, and a description.

AppUser: Represents a registered user of the application. Each user has a unique identifier, a unique username, a unique email address, a hashed password, and a registration timestamp.

Review: Users can create reviews for cocktails they have tried, giving it a 1-5 star rating and a text description. These reviews should be shown on the bottom of the page of the specific cocktail page.

Tool: For complex cocktails/recipes, there may be need for multiple tools or equipment such as muddlers, shakers, strainers, etc.

2.2 Relationships

GlassType → Cocktail: Each Cocktail is served in exactly one GlassType, representing the recommended serving vessel (e.g. Coupe, Highball, Rocks glass). A single GlassType may be associated with zero or many Cocktails. Mandatory on the Cocktail side and optional on the GlassType side.

Cocktail → Recipe: Each Cocktail has exactly one Recipe. Each Recipe belongs to exactly one Cocktail. Mandatory on both sides — a cocktail must have a recipe, and a recipe must belong to a cocktail.

Recipe → Ingredient: A Recipe requires one or more Ingredients. An Ingredient can appear in many different Recipes. RecipeIngredient associates these entities and stores the quantity and unit of measure for each ingredient in a given recipe. Mandatory on the Recipe side and optional on the Ingredient side.

Recipe → Tool: A Recipe may need zero or more Tools (e.g. muddler, shaker, strainer). A Tool can be needed by zero or many Recipes. RecipeTool associates these entities. Optional on both sides — a simple recipe may require no special tools, and a tool may exist in the database before being assigned to any recipe.

Ingredient → IngredientType: Each Ingredient is classified as exactly one IngredientType (e.g. Spirit, Mixer, Garnish). An IngredientType can classify one or more Ingredients. Mandatory on the Ingredient side and optional on the IngredientType side.

Cocktail → Flavor: A Cocktail can be tagged with one or more Flavors. A Flavor can be assigned to zero or many Cocktails. CocktailFlavor associates these entities. Mandatory on the Cocktail side and optional on the Flavor side.

Cocktail → Review: A Cocktail can have zero or many Reviews. Each Review belongs to exactly one Cocktail. Mandatory on the Review side and optional on the Cocktail side — a cocktail may exist without any reviews yet.

AppUser →Review: An AppUser can create zero or many Reviews. Each Review is created by exactly one AppUser. Mandatory on the Review side and optional on the AppUser side — a user may exist without having written any reviews.

AppUser → Cocktail (Favorites): An AppUser can favorite zero or many Cocktails. A Cocktail can be favorited by zero or many AppUsers. UserFavorite associates these entities and records the timestamp of when the cocktail was favorited. Optional on both sides.

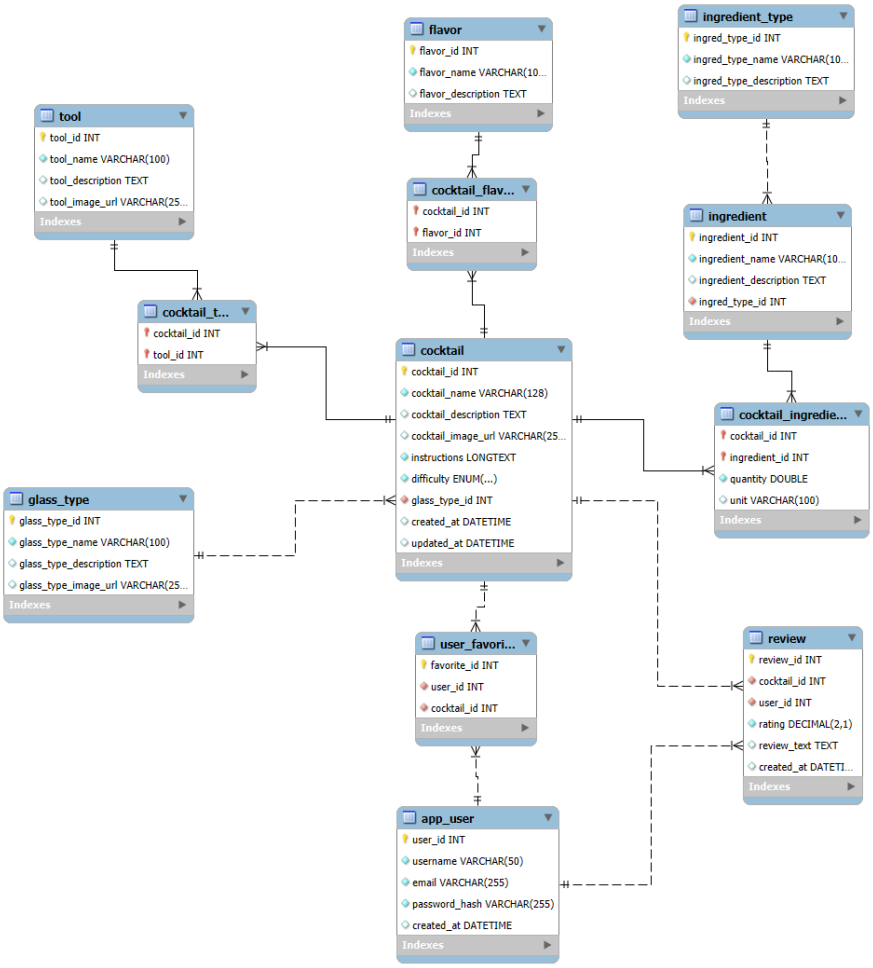

Figure 2: Logical Design

2.3 Proposed Schema

- glass_type(**glass_type_id INTEGER**, glass_type_name VARCHAR(100), glass_type_description TEXT, glass_type_image_url VARCHAR(255))
  - **nullable:** glass_type_description, glass_type_image_url
- ingredient_type(**ingred_type_id INTEGER**, ingred_type_name VARCHAR(100), ingred_type_description TEXT)

  - **AK:** ingred_type_name
  - **nullable:** ingred_type_description

- ingredient(**ingredient_id INTEGER**, ingredient_name VARCHAR(100), ingredient_description TEXT, ingred_type_id INTEGER)
  - **AK:** ingredient_name
  - **FK:** ingred_type_id → ingredient_type(ingred_type_id)
  - **nullable:** ingredient_description
  - **ON DELETE / ON UPDATE:** RESTRICT / RESTRICT

- tool(**tool_id INTEGER**, tool_name VARCHAR(100), tool_description TEXT, tool_image_url VARCHAR(256))
  - **nullable:** tool_description, tool_image_url

- flavor(**flavor_id INTEGER**, flavor_name VARCHAR(100), flavor_description TEXT)
  - **AK:** flavor_name
  - **nullable:** flavor_description

- cocktail(**cocktail_id INTEGER**, cocktail_name VARCHAR(128), cocktail_description TEXT, cocktail_image_url VARCHAR(256), instructions LONGTEXT, difficulty ENUM('Simple','Complex'), glass_type_id INTEGER, created_at DATETIME, updated_at DATETIME)
  - **AK:** cocktail_name
  - **FK:** glass_type_id → glass_type(glass_type_id)
  - **nullable:** cocktail_description, cocktail_image_url
  - **difficulty:** {`Simple`, `Complex`}
  - **created_at:** default CURRENT_TIMESTAMP
  - **updated_at:** default CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
  - **ON DELETE / ON UPDATE (glass_type FK):** RESTRICT / RESTRICT

- cocktail_tool(**cocktail_id INTEGER**, **tool_id INTEGER**)
  - **PK:** (cocktail_id, tool_id)
  - **FK:** cocktail_id → cocktail(cocktail_id)
  - **FK:** tool_id → tool(tool_id)
  - **ON DELETE / ON UPDATE:** CASCADE / CASCADE

- cocktail_ingredient(**cocktail_id INTEGER**, **ingredient_id INTEGER**, quantity DOUBLE, unit VARCHAR(100))
  - **PK:** (cocktail_id, ingredient_id)
  - **FK:** cocktail_id → cocktail(cocktail_id)
  - **FK:** ingredient_id → ingredient(ingredient_id)
  - **nullable:** unit
  - **ON DELETE / ON UPDATE:** CASCADE / CASCADE

- cocktail_flavor(**cocktail_id INTEGER**, **flavor_id INTEGER**)
  - **PK:** (cocktail_id, flavor_id)
  - **FK:** cocktail_id → cocktail(cocktail_id)
  - **FK:** flavor_id → flavor(flavor_id)
  - **ON DELETE / ON UPDATE:** CASCADE / CASCADE

- app_user(**user_id INTEGER**, username VARCHAR(50), email VARCHAR(255), password_hash VARCHAR(255), created_at DATETIME)
  - **AK:** username
  - **AK:** email
  - **password_hash:** store hashed value only
  - **created_at:** default CURRENT_TIMESTAMP
- user_favorite(**favorite_id INTEGER**, user_id INTEGER, cocktail_id INTEGER)
  - **AK:** (user_id, cocktail_id)
  - **FK:** user_id → app_user(user_id)
  - **FK:** cocktail_id → cocktail(cocktail_id)
  - **ON DELETE / ON UPDATE:** CASCADE / CASCADE
- review(**review_id INTEGER**, cocktail_id INTEGER, user_id INTEGER, rating DECIMAL(2,1), review_text TEXT, created_at DATETIME)
  - **AK:** (user_id, cocktail_id)
  - **FK:** user_id → app_user(user_id)
  - **FK:** cocktail_id → cocktail(cocktail_id)
  - **rating:** 0–5 (CHECK constraint)
  - **nullable:** review_text
  - **created_at:** default CURRENT_TIMESTAMP
  - **ON DELETE / ON UPDATE:** CASCADE / CASCADE

2.4 User-Defined Functions, Triggers, Procedures and Events

3 User-Defined Functions (called with `SELECT fn_name(...)`)

- `fn_avg_rating(cocktail_id)` — computes average rating for a cocktail directly in MySQL. The backend calls this when loading cocktail data instead of computing the average in Python. This means the `avgRating` field in the API response comes from the database engine itself.
- `fn_cocktail_review_count(cocktail_id)` — returns review count per cocktail. Used inside `sp_get_cocktail_stats`.
- `fn_user_favorite_count(user_id)` — returns how many favorites a user has. Exposed via the new `GET /api/analytics/user/{id}/favorite-count` endpoint.

4 Procedures (called with `CALL sp_name(...)`)

- `sp_submit_review` — replaces the 4-query review creation (check cocktail exists → check user exists → check duplicate → insert). All validation now happens server-side in one `CALL`. If anything fails, MySQL raises a `SIGNAL` with a readable error message that the backend passes to the frontend.
- `sp_update_review` — replaces the 2-query review update (check exists → update). Same pattern.
- `sp_register_user` — replaces the 3-query registration (check username → check email → insert). Duplicate checking is now atomic in the database.
- `sp_get_cocktail_stats` — returns per-cocktail analytics by calling `fn_avg_rating` and `fn_cocktail_review_coun` internally. Powers the new `GET /api/analytics/cocktail-stats` endpoint and the updated summary endpoint.

4 Triggers (fire automatically, no backend code needed)

- `trg_review_rating_check` (BEFORE INSERT on review) — rejects any review with a rating outside 0–5. This is a safety net on top of the CHECK constraint, since some MySQL versions and tools don't enforce CHECK.
- `trg_review_rating_check_update` (BEFORE UPDATE on review) — same enforcement on edits.
- `trg_cascade_delete_user_reviews` (BEFORE DELETE on app_user) — when a user account is deleted, automatically removes all their reviews. No orphaned data.
- `trg_cascade_delete_user_favorites` (BEFORE DELETE on app_user) — same for favorites.

1 Scheduled Event

- `evt_cleanup_orphan_favorites` — runs daily, removes any favorites or reviews pointing to cocktails that no longer exist. A safety net for referential integrity.

2.5 Lessons Learned & Future Work

- We originally modeled recipe as a separate table, but in practice, recipe had a strict one-to-one relationship with cocktail and no independent lifecycle. That design added unnecessary complexity, including extra joins in queries and a higher risk of data integrity issues (for example, orphaned recipe records or awkward foreign key dependencies). By merging instructions and difficulty directly into the cocktail table, we simplified the schema, reduced query complexity, and made ownership of data clearer. This change improved maintainability and made both backend logic and database initialization easier to reason about. The key takeaway is that normalization should support real domain behavior, and when an entity has no standalone meaning, collapsing it into its parent can produce a cleaner and more reliable design.

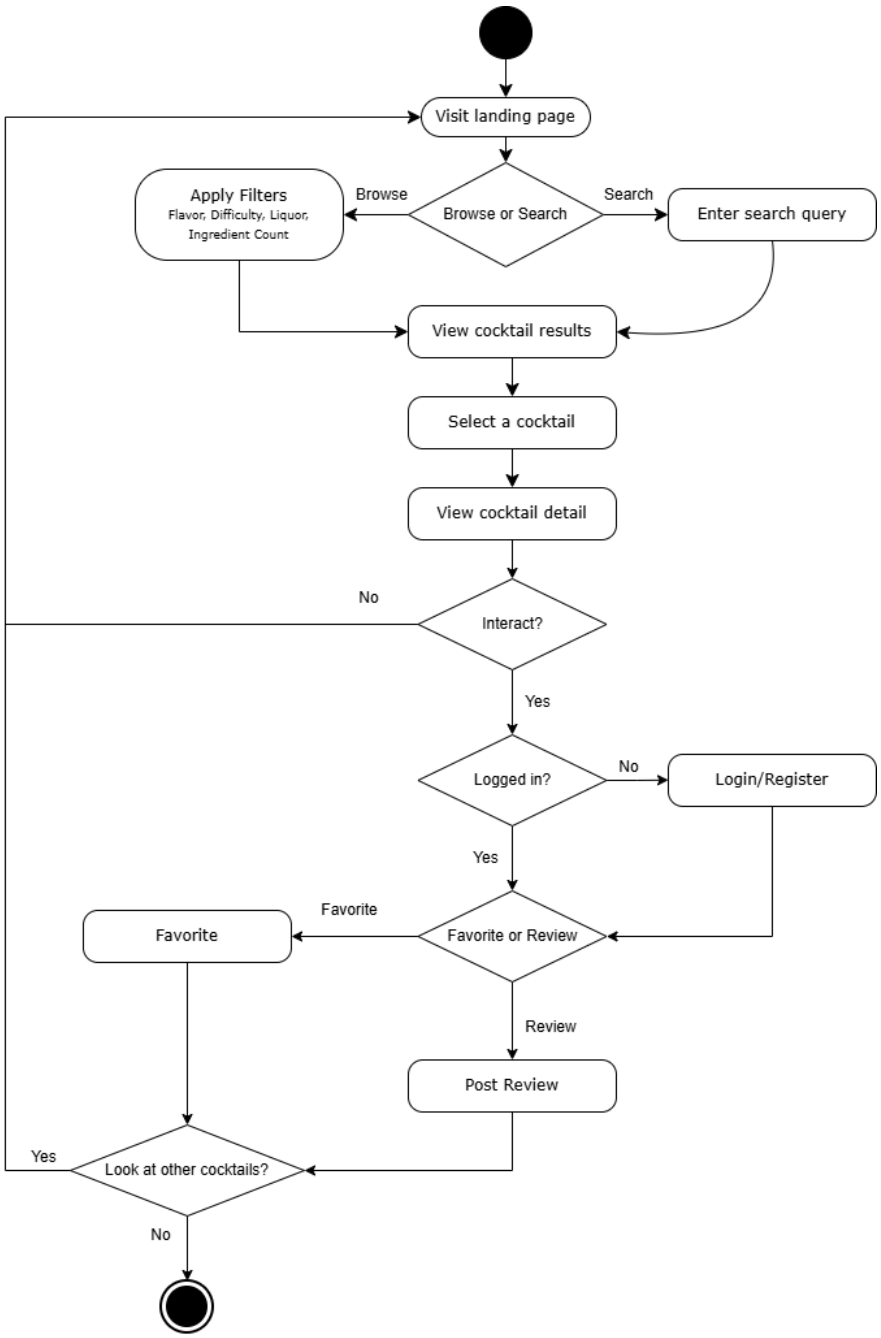

Figure 3: Activity Diagram